

2] When would you use an explicit cursor over an implicit one?

Ans:

When to Use an Explicit Cursor Over an Implicit Cursor

1. When You Need to Retrieve Multiple Rows:

Explicit cursor: If your query returns multiple rows, explicit cursors are necessary because they give you the ability to fetch each row individually and process them as needed.

Implicit cursor: It is best for handling single-row results (e.g., a query that fetches only one row) or simple DML operations (like INSERT, UPDATE, DELETE).

Example: If you want to fetch all employees from a department and print their names, you'd use an explicit cursor.

2. When You Need Greater Control Over the Fetching Process:

Explicit cursor: Provides more control over how rows are fetched. You can decide when to fetch a row, exit when certain conditions are met, and even handle errors or manage logic during the fetch operation.

Implicit cursor: Automatically fetches results, and you have limited control over the fetching process.

Example: When you want to process rows one by one and apply specific logic for each row (e.g., checking for certain conditions before printing or processing the data).

3. For Complex Queries:

Explicit cursor: If your query is complex (e.g., involving multiple joins, unions, or subqueries), an explicit cursor gives you flexibility to manage and manipulate the results more effectively.

Implicit cursor: Best suited for simple queries or one-time operations where a quick result is needed without additional processing.

Example: If you need to retrieve employee names along with their departments and salaries, and then perform additional calculations on these results, you would use an explicit cursor.

4. When You Need to Open, Fetch, and Close the Cursor Manually:

Explicit cursor: You can open and close the cursor manually, giving you the ability to control when the cursor is active. This can be useful if you're working with multiple cursors or need to fetch data in a specific order or condition.

Implicit cursor: It's automatically opened and closed by PL/SQL, and you don't need to worry about managing these operations.

Example: If you need to open a cursor, fetch several rows, process them, and close the cursor at a specific point in the program, explicit cursors are ideal.

5. When You Need to Handle Large Result Sets:

Explicit cursor: Useful when working with large result sets because it allows for row-by-row processing, reducing the need to load the entire result set into memory at once. You can fetch a few rows at a time and process them.

Implicit cursor: It works well for small result sets but can be less efficient with large amounts of data, as it does not allow for manual control over the fetching process.

Example: If you're processing a large number of records, you might fetch records in chunks using explicit cursors.

6. When You Want to Use Cursor Attributes:

Explicit cursor: With explicit cursors, you can use cursor attributes (such as %FOUND, %NOTFOUND, %ROWCOUNT, and %ISOPEN) to track the status of the cursor, such as whether any rows were fetched, how many rows were fetched, or if the cursor is still open.

Implicit cursor: These attributes are not available with implicit cursors.

Example: You might need to check if the cursor has any rows or handle the case where no rows were returned, using explicit cursor attributes.